

# BabyShopHub

## Software Requirements Specification (SRS)

Aptech eProject Submission

Version 1.0

### Table of Contents

|                                                |    |
|------------------------------------------------|----|
| 1. Introduction.....                           | 2  |
| 1.1 Purpose.....                               | 2  |
| 1.2 Product scope.....                         | 2  |
| 1.3 Definitions and abbreviations.....         | 2  |
| 1.4 References.....                            | 3  |
| 2. Overall description.....                    | 4  |
| 2.1 Product perspective.....                   | 4  |
| 2.2 User classes and characteristics.....      | 4  |
| 2.3 Operating environment.....                 | 4  |
| 2.4 Design and implementation constraints..... | 4  |
| 2.5 Assumptions and dependencies.....          | 4  |
| 3. Functional requirements.....                | 6  |
| 3.1 Accounts and authentication.....           | 6  |
| 3.2 Catalogue, search, and reviews.....        | 6  |
| 3.3 Cart, checkout, and payment.....           | 6  |
| 3.4 Orders and tracking.....                   | 7  |
| 3.5 Support.....                               | 7  |
| 3.6 Administration.....                        | 7  |
| 4. Non-functional requirements.....            | 9  |
| 5. Requirements traceability matrix.....       | 10 |

|                                            |    |
|--------------------------------------------|----|
| 6. Principal use cases.....                | 12 |
| UC-1 Register and place a first order..... | 12 |
| UC-2 Track an order to delivery.....       | 12 |
| UC-3 Leave a review.....                   | 12 |
| UC-4 Get help through support.....         | 12 |
| UC-5 Fulfil an order.....                  | 12 |
| UC-6 Manage the catalogue.....             | 12 |
| UC-7 Moderate content and users.....       | 13 |
| 7. Acceptance criteria.....                | 14 |

## 1. Introduction

### 1.1 Purpose

This Software Requirements Specification (SRS) defines what the BabyShopHub application does and the conditions it must satisfy. It is written so that a reader who has never seen the code can understand the full scope of the system, and so that every requirement can be traced to the part of the app that fulfils it. It serves as the agreed baseline between the project brief and the delivered application.

### 1.2 Product scope

BabyShopHub is a mobile shopping application for infant products — diapers, baby food, clothing, bath items, feeding equipment, toys, and related categories. It lets a parent or carer browse a catalogue, search for products, read and write reviews, manage a cart, place an order through a simulated payment step, and track that order to delivery. It gives a shop administrator a back office to manage products, categories, orders, users, reviews, and customer support.

The application runs entirely **on the device**. All data — accounts, products, orders, reviews, and support conversations — is stored in a local SQLite database. There is no server, no external API, and no network call anywhere in the product. This is a deliberate scope decision for the eProject: it keeps the system self-contained, demonstrable offline, and free of any running cost, while still exercising every feature in the brief.

### 1.3 Definitions and abbreviations

| Term     | Meaning                                                    |
|----------|------------------------------------------------------------|
| Customer | A registered shopper using the storefront side of the app. |

|               |                                                                    |
|---------------|--------------------------------------------------------------------|
| Administrator | A privileged user who manages the shop through the admin panel.    |
| SQLite        | A self-contained, file-based database engine embedded in the app.  |
| sqflite       | The Flutter package that provides SQLite on Android and iOS.       |
| Provider      | The state-management approach used to share data across screens.   |
| Dummy payment | A simulated checkout that validates card input but moves no money. |
| Seed data     | Sample products, users, and orders loaded on first launch.         |
| SRS           | This document — Software Requirements Specification.               |

## 1.4 References

The requirements below derive from the Aptech eProject brief for BabyShopHub. The companion documents in this submission are the Windows Setup Guide, the System Design document, the User Guide, the Developer Guide, and the Test Plan.

## 2. Overall description

### 2.1 Product perspective

BabyShopHub is a self-contained, standalone mobile application built with Flutter and Dart. It follows a layered architecture: a presentation layer of screens and reusable widgets, a state layer using the Provider pattern, a repository layer that expresses every data operation as a method, and a data layer that owns the single SQLite database. Because it is standalone, it has no dependency on any external system to function.

### 2.2 User classes and characteristics

The system recognises exactly two classes of user, distinguished by a role stored on the account:

**Customer.** The default role for any account created through registration or the seed data. A customer can do everything on the storefront side but cannot see or reach any administrative function. Customers are assumed to be non-technical parents and carers, so the storefront is designed to be usable without instruction.

**Administrator.** A privileged role, seeded rather than self-registered. An administrator is routed to the admin panel on login and manages the shop's catalogue, orders, users, reviews, and support queue. Administrators are assumed to have basic familiarity with running an online shop.

### 2.3 Operating environment

The application targets Android and iOS mobile devices, and also runs as a Windows or Linux desktop application (used for demonstration when a mobile emulator is not available). It requires no internet connection. Its only storage requirement is the small local database file it creates on first launch.

### 2.4 Design and implementation constraints

The following constraints are fixed for this project:

- The application must be built with Flutter and Dart.
- All persistence must use local SQLite; no cloud service, remote API, or network request may be used.
- Payment must be simulated (dummy), never a real transaction.
- The user interface must follow a single, consistent visual design and be usable by a non-technical audience.
- Passwords must never be stored in plain text.

### 2.5 Assumptions and dependencies

It is assumed that the device can run a current Flutter build, that the user can install the app through the setup guide, and that the seeded demo data is acceptable for

demonstration and grading. The app depends only on a small set of well-established open-source packages (declared in `pubspec.yaml`): `sqlite` for the database, `provider` for state, `crypto` for password hashing, and `path` for locating the database file.

### 3. Functional requirements

Each requirement has a stable identifier (FR-n). The traceability matrix in Section 5 maps every one to the screens and repository that implement it.

#### 3.1 Accounts and authentication

**FR-1 Registration.** A visitor can create a customer account by providing name, email, phone, and a password. The email must be unique and well formed; the password must be at least 8 characters and contain at least one letter and one digit. On success the account is created and the user is signed in.

**FR-2 Login.** A registered user can sign in with email and password. Incorrect credentials are rejected with a clear message. On success, a customer is taken to the storefront and an administrator to the admin panel.

**FR-3 Session persistence.** A signed-in user remains signed in after closing and reopening the app, until they explicitly sign out.

**FR-4 Logout.** Any signed-in user can sign out, which returns the app to the login screen and clears the active session.

**FR-5 Profile management.** A customer can view and edit their profile (name, phone) and manage saved delivery addresses and saved payment cards.

#### 3.2 Catalogue, search, and reviews

**FR-6 Browse by category.** A customer can view all product categories and open any category to see the products within it.

**FR-7 Search.** A customer can search the catalogue by product name or brand and see matching results.

**FR-8 Product details.** A customer can open a product to see its image, description, price, brand, stock status, seller, average rating, and its reviews.

**FR-9 Read reviews and ratings.** A customer can read other customers' star ratings and written reviews on a product, and see the product's average rating.

**FR-10 Write a review.** A customer can leave a star rating and written review on a product. A customer can also rate a seller.

#### 3.3 Cart, checkout, and payment

**FR-11 Add to cart.** A customer can add a product to their cart and choose a quantity.

**FR-12 Manage cart.** A customer can change quantities or remove items, and see the running subtotal, delivery fee, tax, and total. Delivery is free above a configured threshold.

**FR-13 Checkout review.** Before paying, a customer sees an order summary: the items, the delivery address, and the full price breakdown.

**FR-14 Dummy payment.** A customer completes checkout by entering card details that are validated for format only. No real payment is processed. Invalid card input is rejected with a clear message.

**FR-15 Order confirmation.** On successful checkout, the customer sees an order confirmation with the order reference, and the cart is emptied.

### 3.4 Orders and tracking

**FR-16 Order history.** A customer can view a list of their past and current orders with status and total.

**FR-17 Order tracking.** A customer can open an order to see its current status and the history of status changes over time (for example: pending → confirmed → packed → shipped → out for delivery → delivered).

### 3.5 Support

**FR-18 Raise a support ticket.** A customer can open a support ticket by choosing a category, entering a subject, and writing a message.

**FR-19 Support conversation.** A customer can view their tickets and exchange messages within a ticket until it is resolved or closed.

**FR-20 Help content.** A customer can read a set of frequently asked questions and help topics inside the app.

### 3.6 Administration

**FR-21 Admin dashboard.** An administrator sees a summary of the shop: counts and figures that give an at-a-glance picture of activity.

**FR-22 Product management.** An administrator can create, edit, and remove products, including price, stock, description, category, seller, and image.

**FR-23 Category management.** An administrator can create, rename, re-icon, and delete categories. A category that still contains products cannot be deleted.

**FR-24 Order management.** An administrator can view all orders and advance an order's status through its permitted next steps, including cancelling before dispatch.

**FR-25 User management.** An administrator can view all accounts, and suspend or restore a customer's access. The system prevents removing the last active administrator.

**FR-26 Review moderation.** An administrator can view all reviews and hide, restore, or delete a review.

**FR-27 Support handling.** An administrator can view every support ticket, reply to it, and change its status (open, in progress, resolved, closed, or reopened).

## 4. Non-functional requirements

**NFR-1 Responsiveness.** Common interactions — opening a screen, searching, adding to cart — should complete within one to two seconds on a typical device. Because all data is local, no operation waits on a network.

**NFR-2 Usability.** The interface must be intuitive for a non-technical parent. It uses one consistent visual language (colour, spacing, typography), clear labels, empty-state guidance, and confirmation prompts before destructive actions.

**NFR-3 Accessibility.** Text uses legible sizes and sufficient contrast, tap targets are adequately sized, and controls carry meaningful labels. Full conformance to a formal accessibility standard would require dedicated testing with assistive technologies; the app is built to be accessible by construction but is not certified.

**NFR-4 Reliability and error handling.** Every data operation reports success or a human-readable error rather than crashing. Input is validated before it is saved. Invalid states (for example, deleting a non-empty category) are prevented and explained.

**NFR-5 Security.** Passwords are never stored in plain text; each is hashed with SHA-256 using a per-account random salt. Only registered users can reach customer features, and only administrators can reach the admin panel. A suspended account cannot sign in.

**NFR-6 Scalability.** The schema is normalised and indexed by primary and foreign keys, so the catalogue and order history can grow substantially without redesign. The layered architecture allows features to be added without disturbing existing ones.

**NFR-7 Maintainability.** The codebase is organised into clear layers with a single source of truth for pricing, statuses, and theme, so a change is made in one place. It is documented by the companion Developer Guide.

**NFR-8 Portability.** The same code runs on Android, iOS, and desktop; the database layer adapts to each platform automatically.

## 5. Requirements traceability matrix

Every functional requirement maps to the screens that present it and the repository that performs its data work. Screen paths are under lib/screens/ and repositories under lib/data/.

| Req   | Implemented by (screens)                                                         | Data layer                            |
|-------|----------------------------------------------------------------------------------|---------------------------------------|
| FR-1  | auth/register_screen                                                             | auth_repository                       |
| FR-2  | auth/login_screen                                                                | auth_repository                       |
| FR-3  | splash_screen, main (RootGate)                                                   | session_provider                      |
| FR-4  | customer/account_screen, admin/admin_shell                                       | session_provider                      |
| FR-5  | customer/edit_profile, address_list, address_form, payment_methods, payment_form | auth_repository, order_repository     |
| FR-6  | customer/categories_screen, product_list_screen                                  | catalog_repository                    |
| FR-7  | customer/product_list_screen (search)                                            | catalog_repository                    |
| FR-8  | customer/product_detail_screen                                                   | catalog_repository, review_repository |
| FR-9  | customer/product_detail_screen                                                   | review_repository                     |
| FR-10 | customer/write_review_screen                                                     | review_repository                     |
| FR-11 | customer/product_detail_screen                                                   | cart_repository, cart_provider        |
| FR-12 | customer/cart_screen                                                             | cart_repository, cart_provider        |
| FR-13 | customer/checkout_screen                                                         | order_repository                      |
| FR-14 | customer/checkout_screen, payment_form_screen                                    | order_repository                      |
| FR-15 | customer/order_confirmation_screen                                               | order_repository                      |
| FR-16 | customer/orders_screen                                                           | order_repository                      |
| FR-17 | customer/order_detail_screen                                                     | order_repository                      |
| FR-18 | customer/support_new_screen                                                      | support_repository                    |
| FR-19 | customer/support_list_screen, support_detail_screen                              | support_repository                    |
| FR-20 | customer/faq_screen                                                              | faq_data                              |
| FR-21 | admin/admin_dashboard_screen                                                     | admin_repository                      |
| FR-22 | admin/admin_products_screen, admin_product_form_screen                           | admin_repository, catalog_repository  |
| FR-23 | admin/admin_categories_screen                                                    | admin_repository, catalog_repository  |
| FR-24 | admin/admin_orders_screen, admin_order_detail_screen                             | order_repository, admin_repository    |
| FR-25 | admin/admin_users_screen                                                         | admin_repository                      |

|       |                                                            |                    |
|-------|------------------------------------------------------------|--------------------|
| FR-26 | admin/admin_reviews_screen                                 | review_repository  |
| FR-27 | admin/admin_support_screen,<br>admin_support_detail_screen | support_repository |

## 6. Principal use cases

The following use cases describe the main goals users pursue. Each lists the actor, the normal flow, and the outcome.

### UC-1 Register and place a first order

**Actor:** Customer. **Flow:** The visitor registers (FR-1) and is signed in. They browse a category (FR-6) or search (FR-7), open a product (FR-8), and add it to the cart (FR-11). They review the cart (FR-12), proceed to checkout and confirm the address and totals (FR-13), enter card details at the dummy payment step (FR-14), and receive an order confirmation (FR-15). **Outcome:** A new order exists in “pending” status and appears in the customer’s order history.

### UC-2 Track an order to delivery

**Actor:** Customer. **Flow:** The customer opens their order history (FR-16), selects an order, and views its current status and full status timeline (FR-17). **Outcome:** The customer understands where their order is without contacting anyone.

### UC-3 Leave a review

**Actor:** Customer. **Flow:** The customer opens a product they have an opinion on (FR-8) and submits a star rating and comment (FR-10), which then appears among the product’s reviews (FR-9). **Outcome:** The product’s average rating updates and the review is visible to others.

### UC-4 Get help through support

**Actor:** Customer. **Flow:** The customer reads the FAQ (FR-20); if unresolved, they open a ticket (FR-18) and exchange messages (FR-19) until an administrator resolves it. **Outcome:** The customer’s question is answered and the ticket is closed.

### UC-5 Fulfil an order

**Actor:** Administrator. **Flow:** The administrator opens the orders list (FR-24), selects a pending order, and advances its status step by step as it is packed, shipped, and delivered. Each change is recorded in the order’s timeline. **Outcome:** The customer’s tracking view (UC-2) reflects each change.

### UC-6 Manage the catalogue

**Actor:** Administrator. **Flow:** The administrator adds or edits a product (FR-22) and organises categories (FR-23). Empty categories can be removed; categories with products are protected. **Outcome:** The storefront reflects the updated catalogue immediately.

## UC-7 Moderate content and users

**Actor:** Administrator. **Flow:** The administrator hides or deletes an inappropriate review (FR-26) and, where necessary, suspends a customer account (FR-25), which blocks that customer from signing in. **Outcome:** The shop's content and access are kept in good order.

## 7. Acceptance criteria

The application is considered to meet this specification when:

- Every functional requirement FR-1 to FR-27 can be demonstrated end to end using the seeded demo accounts.
- Registration and login enforce their validation rules, and an administrator and a customer are routed to their respective areas.
- A customer can complete UC-1 (register to order confirmation) without error, and the resulting order is trackable through UC-2.
- An administrator can complete UC-5 (advance an order through its full status flow) and the customer's tracking view reflects it.
- Passwords are stored only as salted hashes, and a suspended account cannot sign in (NFR-5).
- The project passes flutter analyze with no issues on a machine with the Flutter SDK, and the app launches to the login screen.

*The Test Plan document defines the concrete test cases that verify these criteria.*